Chapter 7

Bundling

In the beginning, JavaScript was simpler. HTML elements in your markup had attributes with names like `onclick`, and if you wanted something dynamic to happen, you'd write, like, one line of code in the `onclick` attribute. Although this was simple, it was extremely limited. Eventually coders wanted more complex effects in the browser and started placing JavaScript code in files external to the HTML and sending them to the browser via the `script` tag.

This was still sort of simple but also still sort of limited. As time moved on, JavaScript programs got more and more complicated, and the need to include multiple JavaScript files grew. Tools that compiled code to JavaScript appeared. Dependencies on third-party libraries grew. We added CSS, Sass, CoffeeScript, templates, and on and on. And in addition to just compiling, we added performance-related tasks like minifying code or pruning unused CSS selectors.

There started to be a need to coordinate all of these build tasks and make the path from the developer's view of the code to the browser's view of the code clearer.

Rails 7 provides tools that manage this coordination differently than past versions of Rails. The Rails 7 tools draw a sharper line between "things that Rails is responsible for" and "things that the front-end tooling is responsible for," and make the Rails part of that division much simpler.

In earlier versions of Rails, you would likely have used Sprockets or Webpacker to configure and manage front-end tooling. Both of these tools had relatively